Mixed Reality Conversational Support for Older Adults Design, Validation, and Verification

4ZP6-G4

William Clubine
clubinew@mcmaster.ca

Gregory Archer
archeg1@mcmaster.ca

Jasiri Wa-Kyendo
wakyendj@mcmaster.ca

Nicholas Zajkeskovic
zajkeskn@mcmaster.ca

Mayowa Adesanya
adesanym@mcmaster.ca

Khach Kehyayan
kehyayak@mcmaster.ca

Version 0

Supervisors: Dr. Melina Head, Dr. Aydin Farrokhi (DeGroote School of Business)

Repo: https://github.com/Macstone-25/Mixed-Reality

Table of Contents

Table of Contents 1

Purpose Statement 3

Component Diagram 3

Component Information 4

- Frontend (UI) 4
  - Responsibilities and Behaviour 4
  - API 4
  - Implementation Details 4
  - Potential Undesired Behaviours 4
  - Related Requirements 4
  - Test Plan 5
- Speech Recognition Client 5
  - Responsibilities and Behaviour 5
  - API 5
  - Implementation Details 5
  - Potential Undesired Behaviours 6
  - Related Requirements 6
  - Test Plan 6
- Experiment Controller 7
  - Responsibilities and Behaviour 7
  - API 7
  - Implementation Details 7
  - Potential Undesired Behaviours 7
  - Related Requirements 8
  - Test Plan 8
- Trigger Engine 8
  - Responsibilities and Behaviour 8
  - API 9

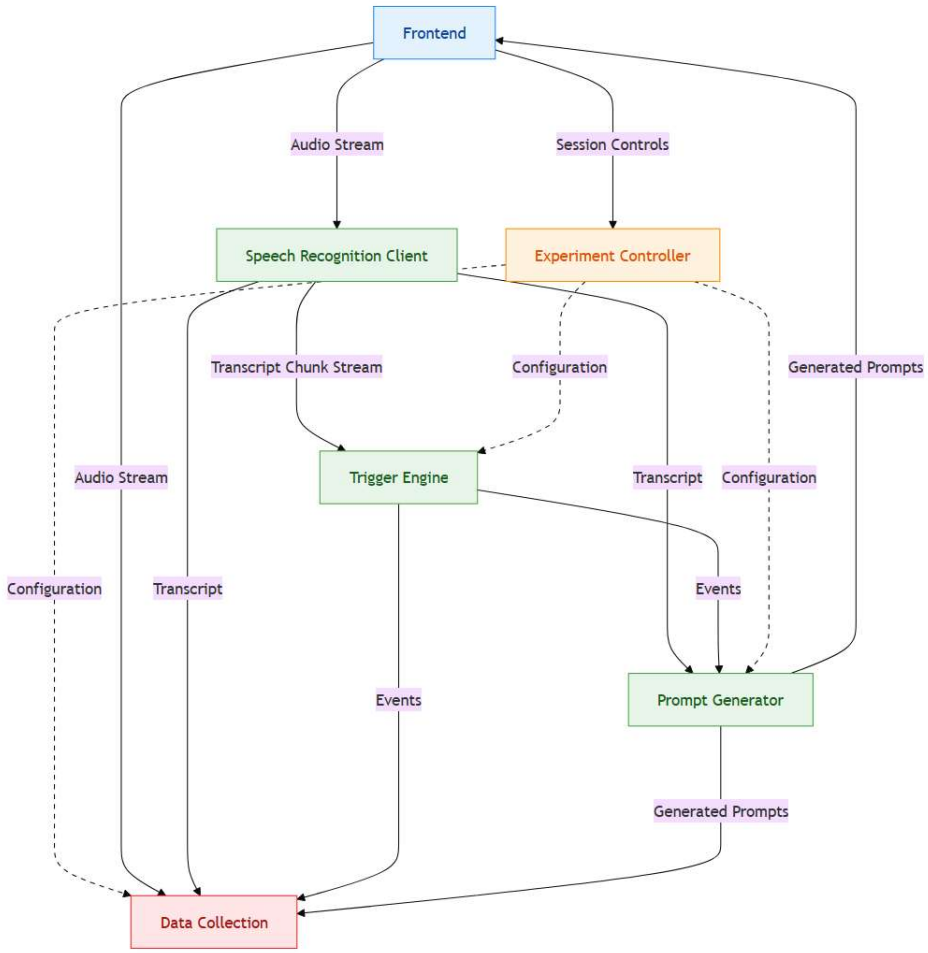
Implementation Details 9
Potential Undesired Behaviours 9
Related Requirements 10
Test Plan 10
Prompt Generator 10
Responsibilities and Behaviour 10
API 10
Implementation Details 11
Potential Undesired Behaviours 11
Related Requirements 11
Test Plan 11
Data Collection 12
Responsibilities and Behaviour 12
API 12
Implementation Details 12
Potential Undesired Behaviours 13
Related Requirements 13
Test Plan 13
User Interface Design 13
Appendix 15
Appendix A: Database Schemas 15
Appendix B: User Interface Mock-ups 16

Please note that this document contains 3 pages of front matter and 2 pages of appendices, bringing the content page count down to 12 instead of the apparent 17.

Purpose Statement

This project builds a real-time conversational support system for older adults on Apple Vision Pro. During a session, the app captures speech, streams a live transcript, detects low-engagement moments (such as extended pauses), and surfaces short, context-aware prompts to help the user continue the conversation; it also records transcripts, triggers, prompts, and configuration for research analysis. This Design, Validation, and Verification (DVV) document describes the Version 0 architecture and how the components work together, including expected behavior, interfaces, implementation details, and potential failure modes, and it defines the test and evaluation plan (functional tests plus performance and effectiveness metrics) that will be used to verify the system meets its requirements and to provide evidence for the final submission.

Component Diagram



Component Information

Frontend (UI)

Responsibilities and Behaviour

Provides the participant-facing layer for the Vision Pro app. It (1) starts sessions only after microphone permission is granted, (2) transitions into the immersive session experience, (3) hosts the in-session HUD (elapsed time + stop button) for session control, and (4) displays AI-generated conversational prompts. For further information on the design, please see the User Interface Design section at the end of this document.

API

All inputs to the frontend UI are passed through the AppModel state object, as is standard for SwiftUI apps of this class. Notable properties of the AppModel include:

- **isSessionActive: Bool** – toggles between the session start view and active session view.
- **Prompt: String** – drives prompt overlay visibility and content.
- **lastSessionError: String?** – displayed on the StartView window when mic permission is denied, session launch fails, or any other unexpected error occurs.
- **immersiveSpaceId, windowGroupId** – used to open/dismiss windows/spaces.

Implementation Details

This component is implemented using **SwiftUI** (view composition + state-driven rendering) and **RealityKit** for an immersive presentation. Core views include:

- **ContentView:** gates the flow (Start → Immersive space)
- **StartView:** "Start Session" and microphone permission handling
- **ImmersiveView:** anchors UI to the user's head and attaches two overlays
    - **SessionControlsView:** elapsed time + stop session button
    - **PromptView:** prompt text

Potential Undesired Behaviours

Microphone permissions blocked: If microphone access is denied, the session cannot start. The UI must clearly direct the user to enable access in Settings (via lastSessionError).

Unresponsive UI: The app has been observed to be significantly more responsive in the simulator than on the actual Vision Pro device, which could lead to participant confusion or frustration.

Related Requirements

- **FR-2:** Provide session controls (start/stop; pause is supported at the session layer if enabled).

- **FR-5:** Surface prompts (visual cue) upon low engagement detection (prompt display).
- **FR-12:** Front-end optimized for accessibility and clarity, reflecting older-adult feedback.

Test Plan

To verify the Frontend UI, we will use a mix of automated and manual testing. Automated XCUITest flows will validate that the Start screen renders correctly, the immersive HUD (SessionControlsView and PromptView) appears during an active session, and the Stop control ends the session cleanly and returns the user to the main window. Because microphone permission prompts are controlled by the operating system and are difficult to automate reliably, microphone-denied and microphone-granted paths will be verified through targeted manual tests, ensuring the correct error messaging is shown when permission is denied and that a session only starts when permission is granted. Finally, XCTest unit tests with a mocked AppModel will validate state-driven UI behavior for prompts, specifically that PromptView visibility is tied to whether prompt is empty.

Speech Recognition Client

Responsibilities and Behaviour

The Speech Recognition Client continuously captures microphone audio and streams it to Deepgram via WebSocket for real-time transcription. It saves transcripts locally and to files for data collection, preserving chunking and timestamps for downstream processing.

API

The Speech Recognition Client does not take inputs directly from other components, but instead is started / stopped by the experiment controller and (when running) processes an incoming audio stream from the device's microphone to output two types of transcripts:

- **Transcript chunks:** Speech fragments generated in real time for use by the Trigger Engine
- **Full transcripts:** Complete sentences, timestamped and diarized (attributed to a particular speaker) for use by the Prompt Generator and in researcher analysis

Implementation Details

This component is implemented in Swift using AVFoundation for audio capture and Starscream for WebSocket communication with Deepgram.

The core classes include SpeechProcessor, which manages audio capture, conversion, and streaming; DeepgramResponse, which models Deepgram's JSON responses; and DeepgramTranscriptChunk, which represents timestamped transcript segments delivered to other components via the SpeechProcessorDelegate class.

Key properties include the audio engine and related nodes for managing audio flow, a WebSocket connection to Deepgram, per-speaker transcript storage, and audio format configuration. Core methods handle audio session setup, audio format conversion, WebSocket events, JSON processing, and resource cleanup.

SpeechProcessor owns the audio engine and WebSocket connection, parses API responses internally, and emits structured transcript chunks to delegates. This design cleanly separates real-time audio capture and transcription from downstream processing.

Potential Undesired Behaviours

- **Inaccurate Transcription:** Errors in transcription can occur due to accents, tone of voice, background noise, or fast speech. The most common transcription error is improper diarization, which results in transcript lines being attributed to the wrong speaker and can lead to confusing prompts being generated.
- **Network Errors:** A stable internet connection is required to get the best transcription results and avoid issues with the WebSocket connection to Deepgram's services.

Related Requirements

- **FR-3:** The application, when directed, shall capture live conversational input and produce a text-formatted transcript of the dialogue
- **FR-7:** The application shall store session artifacts (transcription, processed audio-recordings, timestamps, prompts shown) locally for the purpose of research inspection, statistical analysis, and compliance
- **FR-11:** The application shall upload recorded session data, including transcripts, timestamps, and AI-prompts in a structured CSV or JSON format
- **FR-14:** The application shall automatically anonymize (or pseudonymize) participant data prior to export or storage to ensure compliance with research ethics and data-protection

Test Plan

The Speech Recognition Client is validated through unit and performance testing to ensure reliable audio capture, transcription, and storage. Unit tests cover audio and WebSocket initialization, audio conversion, Deepgram response parsing, transcript chunk delivery, anonymized transcript storage, and graceful handling of network failures.

Performance testing evaluates end-to-end transcription latency, audio drop rate, transcription quality, and long-session stability, to ensure near real-time operation and reliable data collection.

Experiment Controller

Responsibilities and Behaviour

Manages session-level configuration and control settings used during an experimental run. It receives session control inputs from the frontend (start, stop) and applies randomly selected parameters to influence how other components behave during the session. These parameters include, but are not limited to, how prompts are generated, how low-engagement events are handled, and which language model is used.

API

The Experiment Controller API is exposed through the AppModel class and consists of session-level control, shared state, and coordination between the UI, speech processing pipeline, trigger engine, and prompt generator.

- **AppModel():** Initializes the experiment controller and wires the trigger service callback to prompt generation and artifact logging. Creates the PromptGenerator and registers the trigger event handler used during the session.
- **startSession():** Transitions the application into an active session state. Initializes the ArtifactCollector, lazily creates and configures the SpeechProcessor, and begins listening for live audio input.
- **endSession():** Terminates the active session. Tears down the audio engine, resets the trigger service, clears the current prompt, logs the session end, and finalizes artifacts.
- **speechProcessor(chunk: DeepgramTranscriptChunk):** Receives real time transcript chunks from the speech recognition client, logs the transcript data, and forwards it to the trigger engine for processing.

Implementation Details

This component is implemented in Swift within AppModel.swift and serves as the session-level coordination layer. AppModel owns and wires the core runtime components and manages session lifecycle state.

Potential Undesired Behaviours

- **ArtifactCollector initialization failure or misuse**: If ArtifactCollector fails to initialize at session start or is accessed before initialization, logging and artifact storage may fail or trigger a runtime crash.
- **Race conditions during session teardown**: Transcript chunks or trigger events may arrive while endSession() is tearing down the speech processor and finalizing artifacts, potentially causing late prompts, inconsistent logs, or writes to finalized storage.

Related Requirements

- **FR-2:** Provide session control features (start, pause, stop) by coordinating session state and control signals with the frontend and downstream components.
- **FR-15:** Allow researchers to configure experimental scenarios, including selection of LLM provider, AI-support levels, timing of prompt delivery, context window parameters, and other experimental variables.
- **FR-4:** Support detection of low engagement indirectly by supplying configuration parameters (thresholds, timing windows, trigger sensitivity) to the Trigger Engine.
- **FR-10:** Support use of multiple LLM APIs by managing and distributing model-selection configuration to the Prompt Generator.
- **FR-17:** Support computation and aggregation of session metrics by ensuring session configurations are consistently recorded and associated with collected data.

Test Plan

As the Experiment Controller is implemented in Swift within AppModel, testing will focus on validating correct session lifecycle behavior and coordination between components. Unit and component-level tests will be written using XCTest (as standard), where feasible to verify session start/stop behavior, state transitions, and safe teardown of dependencies. In addition, manual and instrumented integration tests will be used to validate that sessions initialize and terminate correctly, that transcript data is forwarded into the trigger pipeline during active sessions, and that UI-visible state (e.g., active session state and prompts) updates as expected. Performance testing will measure session start and stop responsiveness, with a target of < 1 second of latency.

Trigger Engine

Responsibilities and Behaviour

The Trigger Engine detects low engagement during live sessions and emits structured intervention events for downstream prompting and logging. It consumes a stream of TranscriptChunks and maintains a bounded rolling context buffer. As chunks arrive, it updates context and runs a pause rule that arms/reschedules a one-shot silence timer after the last meaningful speech utterance. Filler/trailing filler (e.g., “um”, “uh…”, short fragments) are ignored after a separately configurable duration to ensure that prolonged pauses without silence also receive prompts. If no meaningful continuation occurs for silenceThreshold seconds, the engine fires a pause-based intervention, emits one event per silence episode, and re-arms only after meaningful speech resumes. Each event includes a contextWindow snapshot to keep prompts focused on the current topic.

API

The Trigger Engine API is surfaced to other components through an EventTriggerEngine class, providing the following high-level methods and interfaces:

- **EventTriggerEngine(silenceThreshold: TimeInterval = 4.0, mode: Mode = .ruleBased, llm: LLMEvaluator? = nil, contextWindow: Int = 8)**: Creates a new trigger engine for the session using the timing/context configuration, and wires to PauseRuleTrigger.
- **events: AnyPublisher<InterventionEvent, Never>**: A Combine publisher that emits InterventionEvents for prompt generation, UI, and logging to subscribe to.
- **receive(chunk: TranscriptChunk)**: Adds a transcript chunk to the rolling context buffer and forwards it to the pause rule to update detection state.
- **updateLLMMode(enabled: Bool, llm evaluator: LLMEvaluator?)**: Toggles between .ruleBased and .llmAugmented mode and sets/clears the evaluator without resetting state.
- **reset()**: Clears context and resets pause detection state to a fresh baseline.

Implementation Details

The implementation spans several Swift classes within the Triggers subsystem:

- **Triggers/EventTriggerEngine** coordinates the pipeline: it receives TranscriptChunks, updates the rolling context buffer, forwards chunks to PauseRuleTrigger, and publishes InterventionEvents.
- **Triggers/PauseRuleTrigger** implements “silence after meaningful speech” by tracking the last meaningful speech time, arming/rescheduling a one-shot timer using silenceThreshold, applying lightweight heuristics (contentful vs filler, minimum content, trailing filler), and suppressing repeated firing within the same silence span.
- **Triggers/TranscriptChunk** defines the canonical transcript unit.
- **Triggers/InterventionEvent** defines the exported event payload and reason typing, with optional gating hooks in **Triggers/LLMEvaluator** and **Triggers/HeuristicLLM**.

The engine is then wired into the app via **Services/TriggerDemoService** to be consumed by **Prompts/PromptGenerator** and logging. Timer work runs off the UI thread, with event delivery marshaled to the main thread for UI safety.

Potential Undesired Behaviours

A major risk is false mid-speech prompts caused by ASR batching (gaps between chunks while the user is still talking). Mitigation is treating interim/non-final updates as “speech ongoing” and pushing out the deadline. On the other hand, it is also possible for trigger rules to be too lenient,

causing prompts to come too slowly and reducing the efficacy of the overall system. It is therefore extremely important to test various timing configurations with the Experiment Controller to find the optimal balance between over and under prompting.

Related Requirements

- **FR-4:** The application shall detect indicators of low engagement, such as but not limited to prolonged silence, hesitation, muttering, and non-conversationally relevant speech.
- **FR-5:** Upon detection of low engagement, the application shall surface (with audio or visual elements) a prompt to help continue the conversation.
- **FR-13:** The application shall classify different types of low engagement and adjust its behavior accordingly.
- **FR-17:** The application shall compute and export metrics (such as average stall duration, response latency, and number of AI prompts per session) for statistical analysis, aggregated based on the session configurations.

Test Plan

As the Trigger Engine is written in Swift, the standard XCTest library will be used to unit test EventTriggerEngine and PauseRuleTrigger. We will include at least one test per public API method (receive, reset, updateLLMMode) and for core pause-detection behaviour. Tests will feed synthetic TranscriptChunk sequences and assert on emitted InterventionEvents (reason/type, timing, and attached context), including long pauses, filler-heavy speech, and “single event per silence episode” anti-spam behaviour. Timing tests will use small thresholds and controlled scheduling to keep results deterministic.

Prompt Generator

Responsibilities and Behaviour

The Prompt Generator produces short, context-aware conversational cues when low engagement is detected during a live interaction. It consumes recent transcript data to generate natural prompts that help the participant continue speaking without disrupting conversational flow. The component does not control dialogue; instead, it provides subtle support that preserves conversational agency while enabling consistent, experimentally evaluable behavior.

API

The prompt generator has a very simple API that takes the recent transcript context as an input (bounded window of recent utterances) and returns a single conversational prompt suitable for real-time display as the output.

Implementation Details

- The Prompt Generator receives structured transcript context from the Trigger Engine.
- Prompt generation is **stateless**, relying only on the provided transcript window.
- Prompt style and verbosity are configured externally to support experimental control.
- Generated prompts are short, natural-language cues designed for easy comprehension.
- All prompts are logged alongside timestamps to support post-session analysis.
- The component is designed to be extensible, allowing future prompt strategies or model variations without modifying upstream components.

Potential Undesired Behaviours

- Overly verbose prompts that interrupt conversational flow
- Ambiguous phrasing that lacks clear reference to conversation context
- Repetitive prompting during a single pause if not properly gated upstream
- Prompts that feel directive rather than supportive

Related Requirements

- **FR-5:** The Prompt Generator transforms detected low-engagement events into short, context-aware conversational cues displayed to the user during an active session.
- **FR-9:** Uses a bounded transcript window supplied by the Trigger Engine to generate grounded prompts that reflect recent conversational content.
- **FR-4:** While detection occurs upstream, the Prompt Generator consumes the resulting intervention signals and produces appropriate conversational responses.
- **FR-7:** Generated prompts are logged alongside transcript segments and timestamps to enable post-session analysis and reproducibility.
- **FR-13:** The generator supports multiple prompt strategies (e.g., semantic completion, reminiscence, affective validation), enabling future experimentation & expansion.
- **FR-15:** Prompt behavior can be varied (e.g., support level, prompt type) to support controlled experimentation and evaluation.

Test Plan

The Prompt Generator is validated using structured, repeatable tests in which predefined transcript excerpts are processed to generate conversational prompts under controlled conditions. Each test case specifies a transcript, a selected prompt type, and a support level, with outputs generated deterministically using a fixed random seed to ensure reproducibility. Generated prompts are logged alongside their configuration parameters and reviewed for

relevance, clarity, appropriateness of tone, and consistency with the intended support behavior. This testing approach enables systematic comparison across prompt strategies, supports experimental evaluation of conversational quality, and ensures the component produces stable, context-aware outputs suitable for real-time use in the system.

Data Collection

Responsibilities and Behaviour

The data collection component is responsible for collecting experiment data during conversation sessions, which can later be aggregated and analyzed by researchers. This involves both providing an API for the other components to store the data they collect *("artifacts")* and uploading the collected artifacts to a cloud database for future analysis.

API

The data collection API is surfaced to other components with an **ArtifactCollector** class, providing the following high-level methods for app-wide data collection:

- **ArtifactCollector(id: String):** Input is a string identifier, and output is a new artifact collection implementing the instance methods below.
- **getFileURL(name: String):** Input is a string artifact name, and output is a unique file path (to a non-existent, non-open file) for the artifact data to be written to.
- **getFileHandle(name: String):** Input is a string artifact name, and output is a file handle (Swift's representation of an open file) for the artifact data to be written to.
- **logEvent(type: String, message: String):** Input includes a message type and message content, which are then written to a global messages/logging artifact.
- **finalize():** Closes all open file handles and prevents any more file URLs or file handles from being created, before uploading the complete collection to the cloud.

Implementation Details

Data collection is primarily implemented in Swift with the **ArtifactCollector** class, whose methods, inputs, and outputs are described in detail above. This class will be constructed by the top-level session controller when a new session begins and essentially manages a session-specific folder for the other components to store collected data within. The finalize() method is intended to be called when the session ends and uploads all the collected artifact files to a Supabase Storage bucket. The uploads collected within a single session are grouped together using entries in a Supabase Database table. All these interactions will be conducted using the Supabase Swift SDK and related database schemas can be found in Appendix A.

Potential Undesired Behaviours

This component is dependent on a third-party cloud database provider, Supabase, to ensure normal behaviour. If Supabase experiences any downtime or data loss, the ArtifactCollector could fail to upload some or all of the collected data. This is somewhat mitigated because local copies of the artifact files are not automatically deleted after a session ends, but is still a risk factor to consider. Additionally, certain VisionOS APIs (such as audio recording) necessitate the existence of the getFileURL() method to circumvent the automatic file handle creation and clean-up provided by getFileHandle(). This could lead to errors if components using getFileURL() do not close any open files before the finalize() method is called.

Related Requirements

- **FR-7:** The application shall store session artifacts (transcription, processed audio-recordings, timestamps, prompts shown) locally for the purpose of research inspection, statistical analysis, and compliance.
- **FR-11:** The application shall upload recorded session data, including transcripts, timestamps, and AI-prompts in a structured CSV or JSON format.
- **FR-14:** The application shall automatically anonymize (or pseudonymize) participant data prior to export or storage to ensure compliance with research ethics and data-protection.
- **FR-16:** The application shall support downloading session data in standard research formats (e.g. CSV, XLSX) for external processing, with support for searching and filtering by parameters such as date, LLM used, etc.
- **FR-17:** The application shall compute and export metrics (such as average stall duration, response latency, and number of AI prompts per session) for statistical analysis, aggregated based on the session configurations.

Test Plan

As the data collection component is written in Swift, the standard XCTest library will be used to write unit tests for the ArtifactCollector class. We aim to have at least one unit test for each of the methods described in the API section and will use strategies like golden files to validate that the component is storing artifact data correctly locally and in the cloud.

User Interface Design

When designing the user interface, our primary goal was to maximize simplicity. This is particularly important for our app because many of the users may already be experiencing some form of cognitive decline that would make a complicated interface difficult or impossible to effectively navigate. To accommodate this, we designed the app to have extremely straightforward user flows with minimal unnecessary text and virtually no extra options, such that

the intended next step is almost always very clear. As a result, there are only three buttons within the entire app: start session, end session, and export data. These are visible in the two UI mock-ups found in Appendix B, which collectively display every UI element in the app.

In addition to our self-imposed philosophical design constraints, we also had to design around the practical constraints imposed by visionOS and the SwiftUI framework. For example, we quickly learned that the default mode of user interaction in visionOS, which uses eye tracking to select UI elements, is effectively unusable without requiring users to undergo a lengthy and challenging calibration process. This was not practical for our use case. As a result, we had to ensure that all buttons were both visually large and physically close to the user in 3D space, allowing users to use the alternate and more reliable mode of interaction: physically reaching into the 3D space and “poking” the buttons.

Appendix

Appendix A: Database Schemas

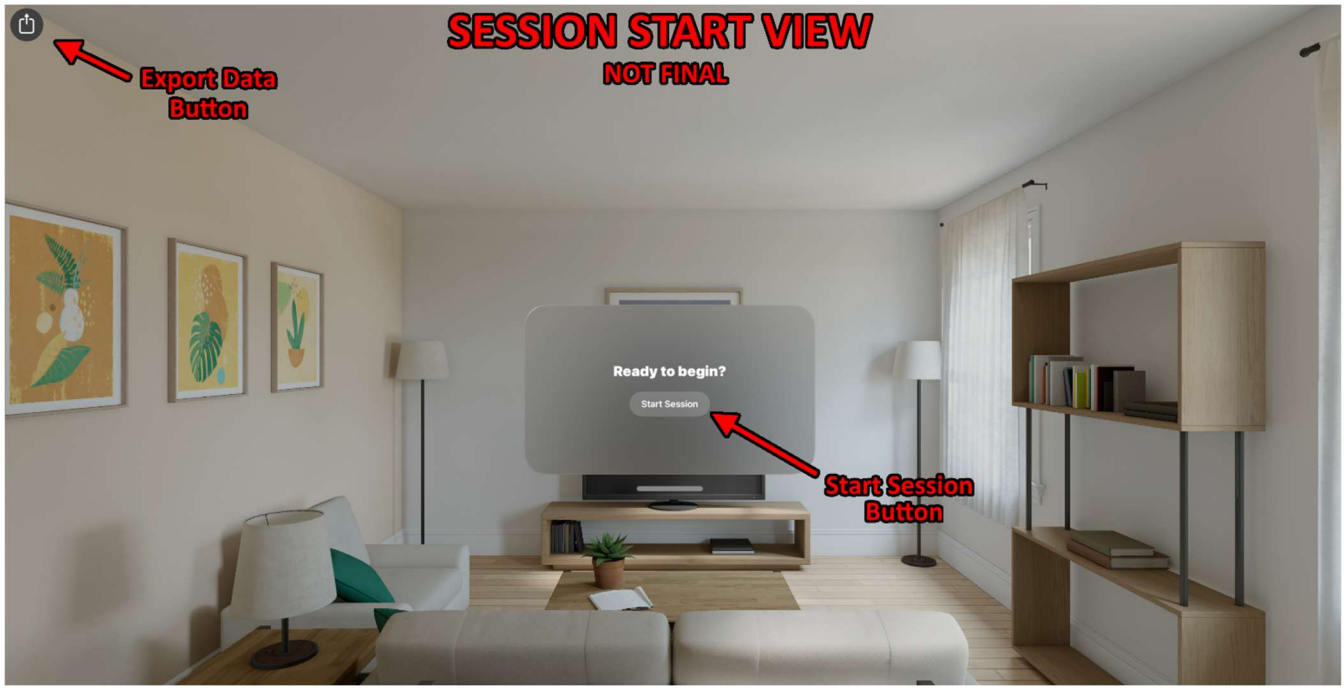
To understand the database columns, it is necessary to understand the naming convention for artifact collections and files:

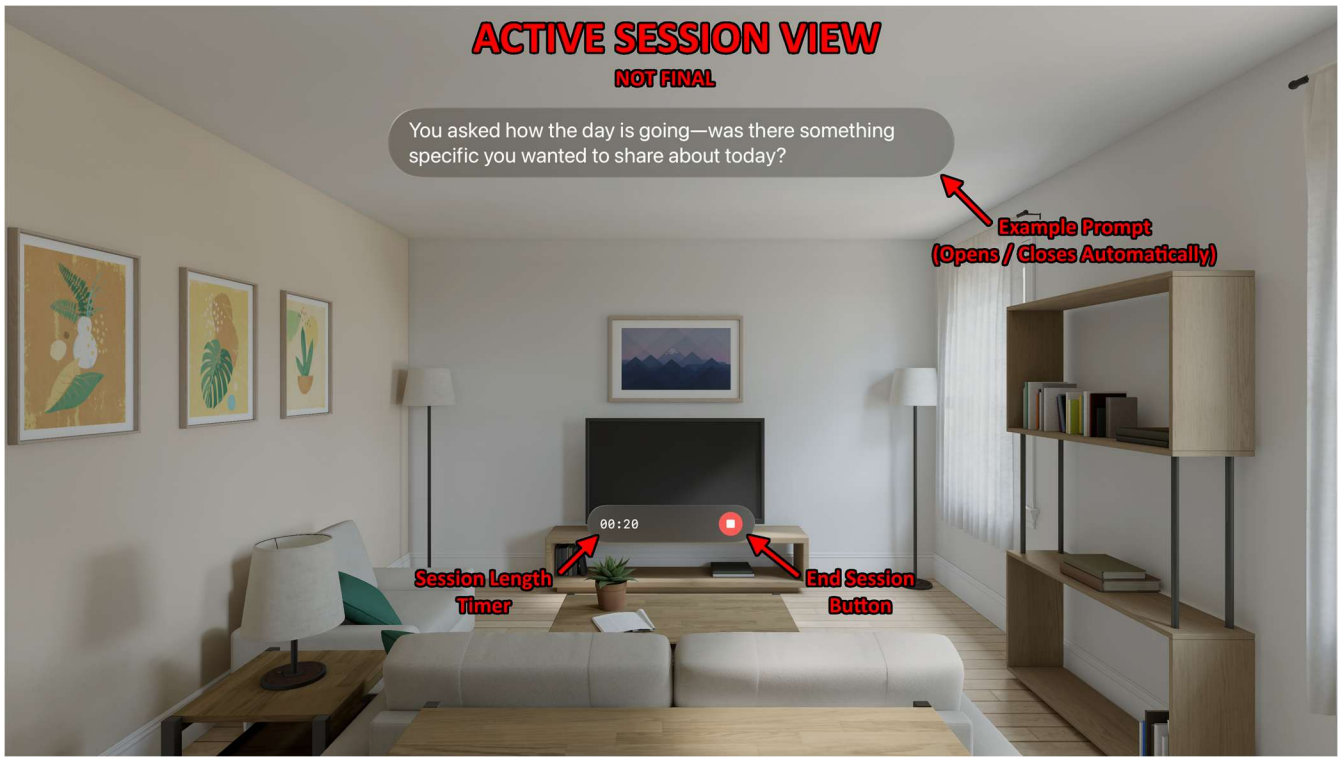
- **Artifact Collection IDs:** (id)-(timestamp)
  - e.g. session-1769141026
- **Artifact File IDs:** (index)-(name)
  - e.g. 0-Events.log, 1-Audio.mp3, 2-Transcript.csv, 3-Configuration.csv, etc.
- **Artifact File Paths (LOCAL):** /(app_storage_path)/(collection_id)/(file_id)
  - The (app_storage_path) is determined by VisionOS
- **Artifact File Paths (CLOUD):** (collection_id)/(file_id)
  - e.g. session-1769141026/0-Events.log
  - Files will be uploaded to a version-named storage bucket (e.g. “artifacts-v0”) so that in the event this naming convention changes, conflicts can be avoided by incrementing the version suffix on the storage bucket name itself

To make it easy to query for all artifacts from a single session or of a specific type, each uploaded artifact is also recorded in a Supabase SQL Database with each component of the upload path (bucket name/version, collection_id, file_id) stored in separate columns:

id int8	uploaded_at timestamptz	version text	collection_id text	file_id text
1	2026-01-23 04:21:12.11674+00	v0	session-1769141026	0-Events.log

Appendix B: User Interface Mock-ups





Please note that the red text has been overlayed to label and provide clarity in the mock-ups but is not present in the actual UI design.